

ASSESSMENT TOOL - 1

Short Answer Test

4. Food Delivery Application

a) Different SDLC Models

1. Waterfall Model

- Development follows sequential phases and requirements are collected first
- Requirements are suitable for fixed requirements
- Easy to understand and manage.

Requirements → Design → coding → Testing → Deployment → Maintain

2. Agile Model

- Development is done in small iterations
- customer feedback is taken after every sprint
- Changes can be made easily

Planning → Design → Development → Testing → Review & Repeat

3. Spiral Model

- Combines development and risk analysis
- suitable for large and complex projects
- Improves software quality and reduces project risks

Planning → Risk Analysis → Development → Evaluation & Repeat

b) Demonstrate the use of Agile practices

- Divide the project into sprints
- Conduct daily standup meetings
- Deliver a working application after every sprint
- Collect customer feedback regularly
- Perform continuous testing
- Fix bugs immediately
- Release updates frequently

PLAN → DEVELOP → TEST → DEPLOY → FEEDBACK

c) Choose and justify a suitable model

Reasons:

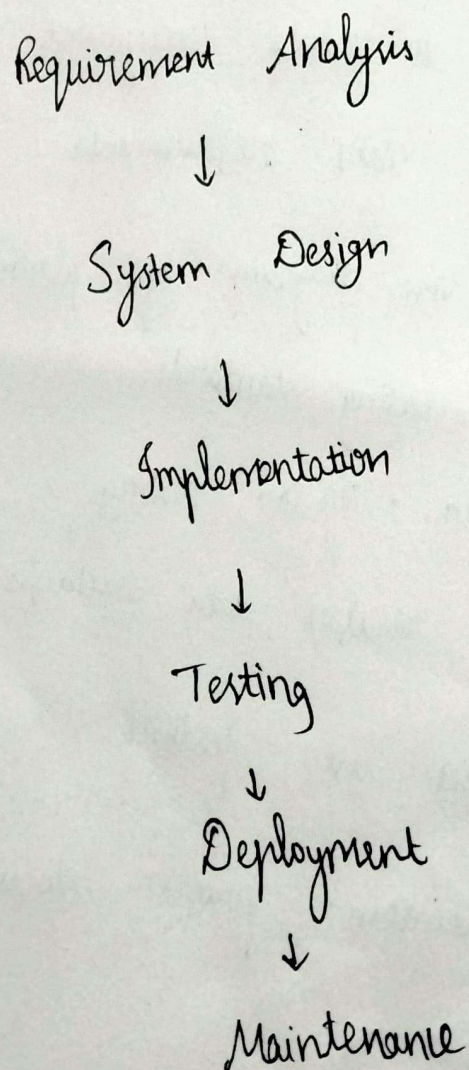
- supports continuous updates
- Handles changing customer requirements
- Delivers software quality
- Improves customer satisfaction
- Easy maintenance
- Better teamwork
- Continuous testing improves quality
- Reduces development risks

3. Student Information System

a) SDLC Phases:

- Requirement Analysis : collect requirements from students
- System Design : Design database, interface, architecture.
- Implementation : Develop modules such as student, faculty
- Testing : Test all modules. Fix errors and bugs
- Deployment : Install the system in the institution
- Maintenance : Update feature. Fix bugs and improve

SDLC DIAGRAM :



b) Show how much phase contributes to structured design

Phase	Contribution
Requirement Analysis	Understands user needs
Design	Create the system structure
Coding	Develops the software modules
Testing	Finds and removes errors

c) Apply Techniques to reduce development risks

- Gather clear requirements
- Use UML diagrams for planning
- Follow coding standards
- Perform regular testing
- Take regular data backups
- Collect user feedback
- Maintain proper documentation